MB Jewellers — Technical Documentation

Document version: 1.0
Last updated: 2 June 2026
Repository: chetansaini0/mb_jewellers
Production target: https://www.mbjewellers.in
Stack: Next.js 16.2.4 · React 19 · TypeScript · Tailwind CSS v4 · Prisma 7 · PostgreSQL

Table of Contents

---

1. Project Overview

1.1 Business Purpose

MB Jewellers is a **premium jewellery showcase and lead-generation website** for a family-run studio in **Sikar, Rajasthan, India**. The site is intentionally **not a full e-commerce platform**: there is no online checkout, cart, or payment gateway. Instead, the digital experience exists to:

- Present gold, diamond, silver, and bridal collections with cinematic, luxury-grade storytelling
- Build trust through heritage narrative, certifications, testimonials, and studio transparency
- Convert visitors into **in-studio appointments**, **WhatsApp conversations**, **contact form inquiries**, and **newsletter subscribers**
- Support multi-language discovery via Google Translate for regional and international audiences

Purchases and valuations happen **in person** at the flagship studio on Mahamandir Road, Chandpol, Sikar.

1.2 Target Audience

Segment	Needs addressed on site
Bridal clients (Rajasthan & NCR)	Curated suites, ceremony-specific styling, private viewing booking
Gold & diamond buyers	Collection browsing by material and subsection (sets, Ranihaar, Rajputi, etc.)
Gifting & silver buyers	Contemporary silver and accessories (serveware, statues)
NRI / diaspora families	Studio location, WhatsApp prefill, trust signals, translation
Local walk-in prospects	Maps embed, hours, phone, directions URL

1.3 User Journey

```
Discovery (SEO / social / referral)
    → Homepage hero + collections hub
    → Collection category OR product detail OR bridal/services
    → Trust (testimonials, promises, heritage)
    → Conversion (contact form / appointment / WhatsApp / newsletter)
    → In-studio visit (offline purchase)
```

Secondary journeys:

- **Admin:** `/admin/login` → session cookie → dashboard for inquiries, appointments, subscriber management
- **Legal/compliance:** Footer links to privacy, terms, refund, shipping, cancellation, cookie, disclaimer policies
- **Content:** Blog articles → product/collection deep links

1.4 Project Objectives

1. Deliver a **world-class luxury web presence** comparable to international jewellery maisons while remaining rooted in Rajasthani heritage
2. Achieve **strong technical SEO** (metadata, schema, sitemap, canonical URLs) for “jewellery in Sikar” and related queries
3. Capture **structured leads** with validation, rate limiting, honeypots, and optional Postgres persistence
4. Maintain **production-grade security** (CSP, HSTS, admin proxy auth, origin checks)
5. Ship a **maintainable codebase** with static generation (~71 routes), CI on GitHub Actions, and Vercel deployment

1.5 Brand Positioning

- **Tone:** Cinematic, restrained, atelier-led (“Where light becomes legacy”)
- **Visual language:** Cream/gold palette, glass cards, Playfair Display + Poppins + Cinzel typography
- **Differentiators:** 30+ years experience, 5 Lakh+ customers (marketing claim), hallmarked gold, certified diamonds, bespoke bridal atelier
- **Geography:** Proudly Sikar-based; maps and `JewelryStore` schema anchor local SEO

---

2. Complete Feature Breakdown

2.1 Homepage (`PremiumHome.tsx`)

The homepage is the primary brand stage, composed of:

Section	Source / component	Description
Hero	`premiumHero` in `premiumContent.ts`	Full-viewport cinematic video (`/instareel/Video-281.mp4`), magnetic CTAs, mouse parallax
Signature worlds	`premiumCollections`	Four category cards (Diamond, Gold, Silver, Accessories) with local cinematic PNGs
Heritage story	`premiumStory`	Three-chapter scroll narrative with GSAP reveals
Promises	`premiumPromises`	Four trust pillars (purity, bespoke, lifetime care, exchange)
Collage	`premiumCollage`	Masonry-style bridal and product imagery

New arrivals	`PremiumNewArrivalsSection`	`featuredPieces` — **9 products** with tilt cards
Testimonials	`premiumTestimonials`	Client quotes with motion
Studio reels	`InstagramReelsSection` (dynamic)	MP4 reels from `/public/instareel/`
Map	`PremiumHomeStudioMapSection` (dynamic)	Google Maps embed for Sikar studio
Trust counters	`PremiumTrustSection`	GSAP animated counters (30+ years, 5 Lakh+ customers)

Heavy client sections use `next/dynamic` with skeleton loaders to protect initial bundle size.

2.2 Collections

Hub: `/collections` — `PremiumCollectionsHubPage`

Category landing pages:

- `/collections/diamond` — 5 subsections (sets, necklaces, rings, earrings, bracelets)
- `/collections/gold` — 9 subsections (sets, Ranihaar, Rajputi, necklaces, chokars, bangles, earrings, chains, rings)
- `/collections/silver` — 7 subsections
- `/collections/accessories` — 2 subsections (statues, utensils)

Dynamic subsection routes: `/collections/[collection]/[section]` — configured in `collectionPages.ts`, statically generated via `buildCollectionSectionParams()` (**23 subsection pages** total).

Configuration is data-driven: each `CollectionPageConfig` defines eyebrow, title, description, and `sections[]` with `id`, `title`, `description`, `coverImage`, `coverAlt`.

2.3 Products

- **Catalog source:** `featuredPieces` in `app/lib/siteData.ts` (**9 items**)
- **Slug mapping:** `premiumProductsBySlug` in `premiumPages.ts` via `slugifyProductName()`
- **Routes:** `/products/[slug]` — e.g. `/products/emerald-diamond-set`, `/products/silver-dew-bracelet`
- **Page component:** `PremiumProductPage` — detail, material tag, imagery, CTAs to contact/WhatsApp

Products are showcase entries, not inventory-backed SKUs (Prisma `Product` model exists for future CMS use).

2.4 Customer-Facing Features

Feature	Route / API	Notes
Contact inquiry	`/contact` + `POST /api/contact`	Zod-style validation in `validation.ts`, honeypot `website` field
Appointments	Forms + `POST /api/appointments`	Types: CONSULTATION, BRIDAL, CUSTOM_DESIGN, REPAIR, OTHER
Newsletter	Footer + `POST /api/newsletter`	Email capture, duplicate handling
WhatsApp	`getWhatsAppUrl()` in `siteConfig.ts`	Prefill messages for appointments
FAQ	`/faq`	Accordion with `aria-expanded` / `aria-controls`
Bridal	`/bridal`	Ceremony moments, dedicated CTAs
Services	`/services`	Custom jewellery steps from `premiumPages.ts`
Heritage / About	`/heritage`, `/about`	Timeline `aboutJourney`
Gallery	`/gallery`	`galleryItems` masonry
Blog	`/blog`, `/blog/[slug]`	**3 articles** in `blogPosts`
Testimonials	`/testimonials`	Social proof page
Language	Footer `FooterLanguageSwitcher`	Google Translate widget; 25+ languages in `siteData.ts`
Cookie consent	`CookieConsent.tsx`	Gates Google Analytics until acceptance
Legal suite	7 policy routes	Content from `legalContent.ts`

2.5 Design & Premium Shell

`PremiumSite.tsx` wraps all non-admin routes:

- `PremiumProviders` (Lenis + GSAP ScrollTrigger sync)
- `PremiumLoader`, `PremiumCursor` (client-only, dynamic)
- `PremiumHeader`, `PremiumFooter`, `PremiumFloatingCtas`
- `PremiumBackdropLogo`, `SkipToContent`
- Admin routes bypass chrome for minimal layout

Legacy note: `DPHomePage` / older `Header` exist but are unused; premium shell is canonical.

2.6 Admin & Lead Management

- **Auth:** `proxy.ts` middleware matcher for `/admin/*` and `/api/admin/*`
- **Session:** Signed cookie via `admin-auth.ts` (`ADMIN_SESSION_SECRET`)
- **Dashboard:** `AdminDashboardClient.tsx` — list/update inquiries and appointments
- **Storage:** `lead-store.ts` — dual mode:
  - `LEAD_STORAGE_MODE=json` → `.data/leads.json`
  - `LEAD_STORAGE_MODE=postgres` → Prisma models `Inquiry` , `Appointment` , `NewsletterSubscriber`

2.7 Data & Content Files (Key)

File	Role
`app/lib/siteData.ts`	Products, categories, social links, flagship studio, translation languages
`app/lib/premiumContent.ts`	Hero, story, promises, collage, testimonials
`app/lib/collectionPages.ts`	Collection/subsection configs
`app/lib/premiumPages.ts`	Blog, gallery, trust values, product slugs, contact channels
`app/lib/seo.ts`	`createPageMetadata` , JSON-LD schemas
`app/lib/siteConfig.ts`	URL resolution, contact, WhatsApp helpers
`prisma/schema.prisma`	Full domain schema (User, Product, Lead, Blog, etc.)

---

3. Technology Stack Analysis

Every dependency is declared in `package.json` . This project does **not** use shadcn/ui, Radix UI, or a component library abstraction—UI is custom Tailwind with a small `app/components/ui/button.tsx` .

3.1 Production Dependencies

Package	Version	Role in MB Jewellers
next	16.2.4	App Router, SSG/SSR, API routes, `next/image` , metadata API, `opengraph-image.tsx`
react	19.2.4	UI rendering, concurrent features, client components
react-dom	19.2.4	DOM bindings

typescript	^5 (dev)	Strict typing across app and lib
tailwindcss	^4 (dev)	Utility-first styling via `@tailwindcss/postcss`
framer-motion	^12.23.12	Hero parallax, magnetic links, page transitions, tilt
gsap	^3.13.0	ScrollTrigger reveals, counters, timeline animations
lenis	^1.3.11	Smooth scroll; integrated with ScrollTrigger in `PremiumProviders`
lucide-react	^1.14.0	Icon set (optimized via `optimizePackageImports`)
@prisma/client	^7.8.0	ORM for Postgres lead/product schema
zod	^4.4.3	Available for schema validation (forms also use custom parsers in `validation.ts`)

3.2 Development Dependencies

Package	Version	Role
prisma	^7.8.0	CLI migrate, generate, studio
eslint	^9	Linting with `eslint-config-next` 16.2.4
prettier	^3.8.3	Formatting (`format`, `format:check`)
@types/node, @types/react, @types/react-dom	^19/^20	Type definitions
@tailwindcss/postcss	^4	Tailwind v4 PostCSS pipeline

3.3 Scripts

Script	Purpose
`dev`	`next dev -H 0.0.0.0 --webpack` — LAN-friendly dev
`build`	Production build (~71 routes)
`start`	Production server
`lint`	ESLint
`typecheck`	`tsc --noEmit`
`verify:build`	Post-build artifact checks (`scripts/verify-production.mjs`)
`postinstall`	`prisma generate`
`prisma:*`	generate, migrate, studio

3.4 Infrastructure Integrations (via env, not npm)

- **Vercel** — hosting, edge, image CDN
- **PostgreSQL** — `DATABASE_URL` for Prisma
- **Upstash Redis** — distributed rate limits (`UPSTASH_REDIS_REST_*`)
- **Resend** — transactional email for new leads (`RESEND_API_KEY`)
- **Google Analytics 4** — consent-gated (`NEXT_PUBLIC_GA_MEASUREMENT_ID`)

3.5 Why This Stack (Design Rationale)

- **Next.js 16 App Router:** File-based routing, static marketing pages, colocated API routes, built-in SEO metadata
- **React 19:** Latest stable with improved hydration patterns for extension-safe forms
- **Tailwind v4:** Rapid luxury UI iteration without CSS module sprawl
- **Framer Motion + GSAP:** Motion for interaction (Framer) + scroll choreography (GSAP)— industry pattern for award-style sites
- **Lenis:** Premium smooth scroll expected in luxury vertical
- **Prisma 7:** Type-safe persistence path from JSON file storage to production Postgres without rewriting business logic

---

4. Frontend Architecture

4.1 Folder Structure

```
cursor/
├── app/
│   ├── layout.tsx              # Root fonts, metadata, JSON-LD, PremiumSite wrapper
│   ├── page.tsx                # Homepage → PremiumHome
│   ├── globals.css             # Design tokens, premium theme
│   ├── components/
│   │   ├── premium/            # Primary UI system (Home, Site, Header, pages/*)
│   │   ├── admin/              # Dashboard client components
│   │   ├── analytics/          # GoogleAnalytics
│   │   ├── a11y/               # SkipToContent
│   │   └── ui/                 # button.tsx (minimal)
│   ├── lib/                    # siteData, seo, lead-store, validation, etc.
│   ├── hooks/                  # useClientMounted
│   ├── api/                    # contact, appointments, newsletter, admin/*
│   ├── collections/            # Category + dynamic [collection]/[section]
│   ├── products/[slug]/        # Product detail SSG
│   ├── blog/[slug]/            # Blog articles
```

```
│   ├── admin/                  # Login + dashboard
│   ├── sitemap.ts, robots.ts, manifest.ts
│   ├── opengraph-image.tsx     # Generated OG image
│   ├── error.tsx, not-found.tsx, loading.tsx
│   └── [legal & marketing pages]/
├── prisma/schema.prisma
├── public/                     # Logo, pics/, instareel/, favicon assets
├── scripts/verify-production.mjs
├── proxy.ts                    # Admin auth middleware (Next.js 16 convention)
├── next.config.ts
└── .github/workflows/ci.yml
```

4.2 Component Architecture

```
RootLayout
└── PremiumSite
    ├── PremiumProviders (Lenis + GSAP)
    ├── PremiumLoader / PremiumCursor (dynamic, ssr: false)
    ├── PremiumHeader
    ├── <main id="main-content">{page}</main>
    ├── PremiumFooter
    ├── PremiumFloatingCtas
    └── CookieConsent

Page routes → premium/pages/*Page.tsx OR PremiumHome
```

Pattern: Marketing pages are thin `page.tsx` files exporting `metadata` + a single premium page component. Shared chrome lives in `PremiumSite`; page-specific content pulls from `lib/` data modules.

4.3 Routing Model

Type	Example	Generation
Static	`/about`, `/faq`	○ Static at build
SSG dynamic	`/products/[slug]`	● `generateStaticParams` from `featuredPieces`
SSG dynamic	`/collections/[collection]/[section]`	● 23 params from `buildCollectionSectionParams()`
SSG dynamic	`/blog/[slug]`	● 3 blog slugs
Dynamic server	`/api/*`, `/admin`	ƒ On demand

Special	`/sitemap.xml`, `/robots.txt`, `/opengraph-image`	Metadata routes

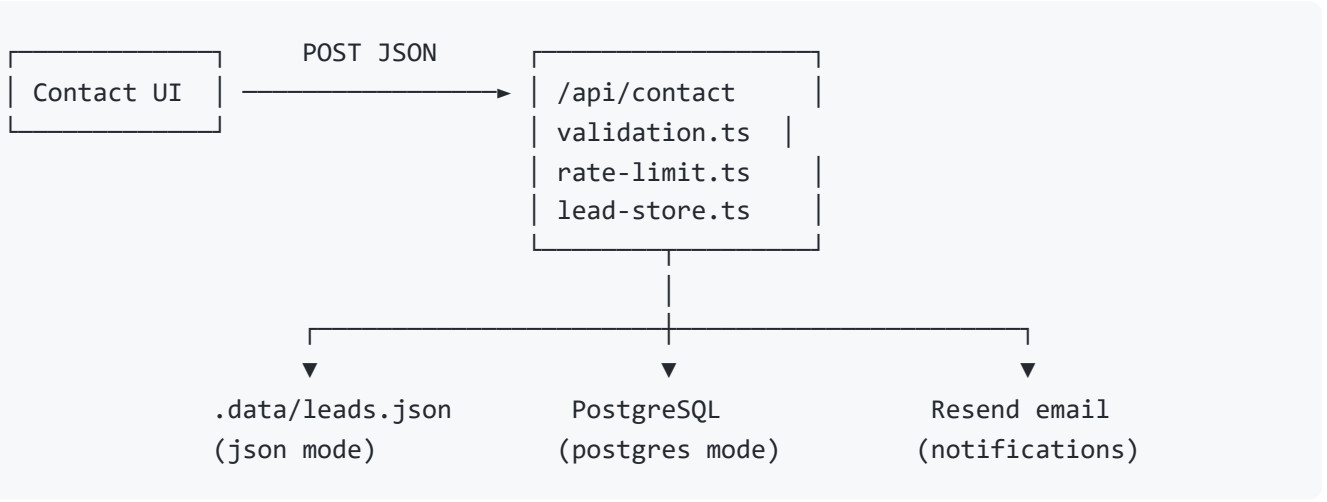
Approximate route count: ~71 prerendered/static paths at build (per `PRODUCTION_READINESS.md` and production build output).

4.4 State Management

No Redux/Zustand. State is localized:

- **React** `useState` / `useRef` — forms, loaders, mobile nav
- **Framer** `useMotionValue` / `useSpring` — hero parallax, magnetic buttons
- **URL state** — Next.js `usePathname` for active nav (`aria-current="page"`)
- **Cookies** — admin session, cookie consent preference (`cookieConsent.ts`)
- **Server persistence** — leads via `lead-store.ts` + Prisma

4.5 Data Flow



Read path (marketing): `siteData.ts` / `premiumContent.ts` → imported at build time → static HTML.

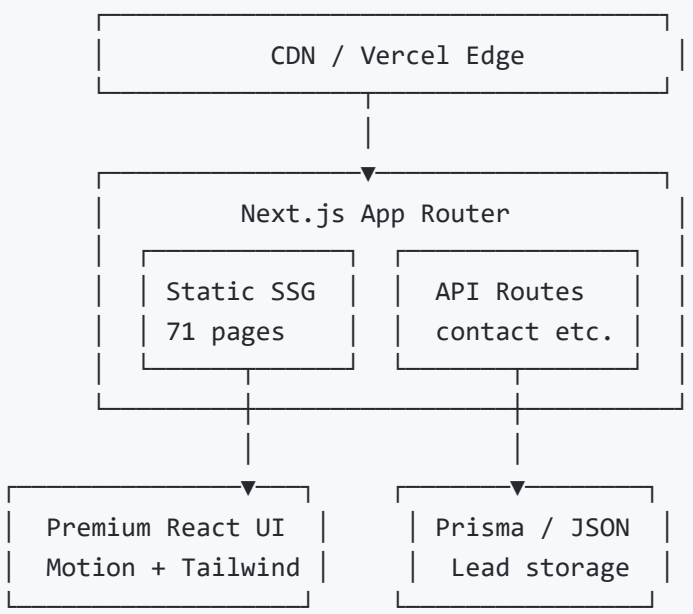
Admin read path: Server components/API routes → `lead-store` aggregations → dashboard tables.

4.6 Client / Server Boundaries

- `"use client"` on motion-heavy components (`PremiumHome` , `PremiumSite` , forms with interaction)
- **Server Components** default for `page.tsx` wrappers that only pass metadata and render client children

- `useClientMounted` hook prevents hydration mismatches for browser-only APIs
- **Dynamic** `ssr: false` for cursor, loader, and optional map/reels

4.7 Text Architecture Diagram



---

5. Animation & Motion Analysis

5.1 Motion Stack Overview

Layer	Library	Primary use
Smooth scroll	Lenis	`PremiumProviders` — RAF loop, `ScrollTrigger.update` on scroll
Scroll reveals	GSAP ScrollTrigger	`usePremiumReveal`, section entrances, blur-to-sharp
Counters	GSAP	`usePremiumCounter` on trust stats
Interaction	Framer Motion	Hero parallax, magnetic CTAs, `PremiumTiltCard`
Custom cursor	Client component	`PremiumCursor` (desktop enhancement)
Page loader	`PremiumLoader`	Initial brand reveal

5.2 Key Implementation Details

Lenis + GSAP integration (`PremiumProviders.tsx`):

- Registers `ScrollTrigger` once

- Adds `lenis-active` class on `<html>`
- On unmount: destroys Lenis, kills all ScrollTrigger instances

Reduced motion (`premiumMotion.tsx`):

- `prefersReducedMotion()` short-circuits GSAP animations
- Lenis does not initialize when `prefers-reduced-motion: reduce`

Hero (`PremiumHome.tsx`):

- Mouse position drives `useTransform` parallax on video/media
- `MagneticLink` uses spring physics on CTA hover

Cleanup:

- `gsap.context()` + `ctx.revert()` prevents memory leaks on route changes

5.3 Performance Considerations

Concern	Mitigation
Main-thread scroll	Lenis RAF; avoid duplicate scroll listeners
Mobile GPU	Cursor/loader disabled or reduced on small viewports (review UX)
Video hero	`preload="metadata"`, poster; MP4 in `/public/instareel/`
Bundle size	`optimizePackageImports` for framer-motion, gsap, lucide-react
Layout thrashing	Transform-only animations where possible

5.4 Accessibility & Motion

- Partial `prefers-reduced-motion` support—**recommendation:** disable hero video autoplay and Lenis when reduced motion is preferred
- Focus states remain visible (`:focus-visible` in global CSS)
- Animated content should not trap keyboard focus (accordions use proper ARIA)

---

6. UI/UX Design Analysis

6.1 Visual System

- **Colors:** Warm cream background (`#faf7f0` theme), gold accents, deep text for contrast

- **Typography:**
  - **Playfair Display** — display headlines (`--font-display-luxury`)
  - **Poppins** — body/UI (`--font-sans-luxury`)
  - **Cinzel** — accent/eyebrows (`--font-cinzel`)
- **Components:** `premium-glass-card` , `premium-eyebrow` , `site-px` / `site-max` layout utilities
- **Imagery:** Mix of brand assets (`/pics/` , `/pics/signature-worlds/`) and Unsplash placeholders

6.2 Layout Patterns

- **Max-width container** with responsive horizontal padding (`site-max` , `site-px`)
- **Clamp-based typography** for fluid scaling across mobile → desktop
- **Footer:** Multi-column grid (5 columns at `xl`) with policies, collections, contact
- **Floating CTAs:** Persistent WhatsApp / contact on scroll

6.3 UX Strengths

1. Clear **conversion paths** without fake e-commerce friction
2. **Studio-first** messaging (private viewing, not “Add to cart”)
3. **Trust density** near decisions (promises, counters, testimonials)
4. **Skip to content** and semantic landmarks for accessibility
5. **Cookie consent** before analytics—GDPR-aligned pattern

6.4 UX Improvement Opportunities

1. Sticky mobile CTA bar for “Book private viewing”
2. Breadcrumb navigation on deep collection pages
3. Replace placeholder Unsplash images with studio photography progressively
4. Explicit “No online checkout—in-store purchase only” near product CTAs
5. Full keyboard audit on mobile nav drawer before launch

---

7. SEO Analysis with Improvements

7.1 Current Implementation

Area	Implementation	File(s)
Title template	`%s | MB Jewellers`	`app/layout.tsx`

Per-page metadata	`createPageMetadata()`	`app/lib/seo.ts`
Canonical URLs	`alternates.canonical`	`seo.ts` + `siteConfig.url`
Open Graph / Twitter	Per-page + root defaults	`seo.ts`, `layout.tsx`
`metadataBase`	From `siteConfig.url`	`layout.tsx`
`robots.ts`	Allow `/`, disallow `/api/`, `/admin/`	`app/robots.ts`
`sitemap.ts`	Static + collections + 23 subsections + 9 products + 3 blogs	`app/sitemap.ts`
JSON-LD	`WebSite`, `JewelryStore` (hours, geo, priceRange)	`layout.tsx`, `seo.ts`
FAQ schema helper	`createFaqPageSchema()`	`seo.ts` (ready for `/faq`)
OG image	Dynamic `opengraph-image.tsx`	App Router image route
Locale	`en_IN` Open Graph locale	Metadata
Admin	`noindex` via `admin/layout.tsx`	Prevents index bloat

7.2 Keyword Strategy

Primary: MB Jewellers, jewellery in Sikar, bridal jewellery Rajasthan, gold jewellery, diamond jewellery, silver jewellery, custom jewellery.

Long-tail: Rajputi gold, Ranihaar, private viewing Sikar, hallmark gold Sikar.

7.3 Implemented Strengths

- Comprehensive sitemap including legal pages (trust signals for Google)
- Local business schema with address matching Google Maps
- `sameAs` links to Instagram and Facebook
- Production URL enforcement on Vercel (`NEXT_PUBLIC_SITE_URL` required)

7.4 Recommended Improvements

Priority	Action	Expected impact
P0	Submit sitemap in Google Search Console after DNS live	Indexation
P0	Set real `NEXT_PUBLIC_GA_MEASUREMENT_ID`	Measurement
P1	Add `FAQPage` JSON-LD on `/faq` using `createFaqPageSchema`	Rich results

P1	`BreadcrumbList` on collection/product pages	SERP clarity
P1	Replace generated OG with brand photo 1200×630 WebP <200KB	CTR
P2	Descriptive `alt` audit on all `next/image`	Image search + a11y
P2	`hreflang` only if multi-locale content (not just Translate widget)	International
P2	Blog publishing cadence for fresh `lastModified`	Crawl frequency

---

8. Performance Analysis

8.1 Build & Delivery

- **~71 static/SSG routes** — fast TTFB on Vercel edge
- **`compress: true`**, **`poweredByHeader: false`** in `next.config.ts`
- **`next/image`** optimization in production (`unoptimized` only in dev)
- **Remote images:** `images.unsplash.com` allowlisted

8.2 Code Splitting

Dynamic imports on homepage:

- `PremiumNewArrivalsSection`
- `InstagramReelsSection`
- `PremiumHomeStudioMapSection`
- `PremiumLoader`, `PremiumCursor` (global, client-only)

8.3 Font & LCP

- Google fonts with `display: "swap"`
- Logo and hero assets use `priority` / `fetchPriority` where configured
- Hero video: metadata preload; poster recommended for LCP stability

8.4 Bundle Optimization

```
experimental: {
  optimizePackageImports: ["framer-motion", "lucide-react", "gsap"],
}
```

8.5 Risks & Monitoring

Risk	Mitigation
Large MP4 reels	Lazy load section; consider `prefers-reduced-motion` gate
GSAP + Lenis on low-end Android	Feature-detect or simplify on mobile
Third-party maps/scripts	CSP already whitelists Google domains
No RUM yet	Enable Vercel Speed Insights post-launch

Target: Lighthouse 90+ on production URL (validate after deploy, not localhost).

---

9. Security Analysis

9.1 HTTP Security Headers (Production)

Configured in `next.config.ts` for all routes:

- `Strict-Transport-Security` (2-year, preload)
- `Content-Security-Policy` (restrictive default-src; GA/maps exceptions)
- `X-Frame-Options: DENY`
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `Cross-Origin-Opener-Policy: same-origin`
- `Permissions-Policy` disables camera/mic/geolocation

9.2 Application Security

Control	Location
Admin route protection	`proxy.ts` + signed session cookie
API origin validation	`request-security.ts` → `isAllowedRequestOrigin`
Rate limiting	`rate-limit.ts` — in-memory fallback; Upstash when configured
Honeypot	`website` field on forms — silent success for bots
Input validation	`validation.ts` — length limits, email/phone regex
Secrets	`.env*` gitignored; `.env.example` without real secrets
Admin noindex	`admin/layout.tsx` metadata

9.3 Data Layer

- Prisma parameterized queries (no raw SQL in hot paths)
- Lead PII stored in Postgres or local JSON—encrypt DB at rest on provider
- `ipHash` field available on `AnalyticsEvent` model for privacy-preserving analytics (future)

9.4 Known Tradeoffs

- CSP includes `'unsafe-inline'` for styles/scripts (maps, inline hydration)—monitor with `report-uri` when available
- Admin password compared to env plain text—**rotate strong password**; bcrypt noted as future improvement
- CI runs `npm audit` — high severity in Next.js tracked; critical gate enforced

---

10. Deployment & Hosting (Vercel)

10.1 Repository & CI

- **GitHub:** `chetansaini0/mb_jewellers`
- **Default branch:** `master` (also accepts `main` in workflow)
- **CI workflow:** `.github/workflows/ci.yml`

Job	Steps
quality	`npm ci`, `prisma generate`, lint, typecheck, format:check
security	`npm audit` (critical fails; high reported)
build	`npm run build`, `npm run verify:build`

Concurrency: cancel in-progress runs on new push.

10.2 Vercel Deployment Flow

1. Import GitHub repository in Vercel dashboard
2. Framework preset: **Next.js**
3. Production branch: `master`
4. Copy all variables from `.env.example` to Vercel Environment Variables
5. **Required:** `NEXT_PUBLIC_SITE_URL=https://www.mbjewellers.in`
6. Set `DATABASE_URL`, run `prisma migrate deploy` against production DB

7. Configure Upstash + Resend for production-grade limits and email
8. Deploy preview → smoke test forms and admin login
9. Attach custom domain `mbjewellers.in` / `www.mbjewellers.in`
10. Verify SSL and promote to production

10.3 Environment Variables (Summary)

Variable	Purpose
`NEXT_PUBLIC_SITE_URL`	Canonical URLs, sitemap, OG
`DATABASE_URL`	Postgres for Prisma
`LEAD_STORAGE_MODE`	`json` (dev) or `postgres` (prod)
`ADMIN_EMAIL`, `ADMIN_PASSWORD`, `ADMIN_SESSION_SECRET`	Admin auth
`UPSTASH_REDIS_REST_URL/TOKEN`	Distributed rate limit
`RESEND_API_KEY`, `RESEND_FROM_EMAIL`, `LEADS_NOTIFICATION_EMAIL`	Lead email
`NEXT_PUBLIC_GA_MEASUREMENT_ID`	Analytics (consent-gated)
`NEXT_PUBLIC_*_CONTACT_*`	Phone, email, WhatsApp

10.4 Post-Deploy Verification

- `npm run build` locally mirrors CI
- Hit `/robots.txt`, `/sitemap.xml`, `/manifest.webmanifest`
- Test `POST /api/contact` from production origin only
- Confirm admin redirect to `/admin/login` when unauthenticated

---

11. Resume Preparation

11.1 ATS-Friendly Bullet Points

Use these as-is or tailored per job description:

1. Architected and developed a **71-route** luxury jewellery marketing site using **Next.js 16 App Router**, **React 19**, and **TypeScript**, achieving static generation for SEO-critical pages.
2. Built a **premium motion system** combining **Framer Motion**, **GSAP ScrollTrigger**, and **Lenis** smooth scroll with `prefers-reduced-motion` safeguards and proper animation cleanup.

3. Implemented **lead-capture APIs** (contact, appointments, newsletter) with **rate limiting**, honeypot bot protection, **Zod-ready validation**, and dual **JSON/PostgreSQL** storage via **Prisma 7**.
4. Designed **admin authentication middleware** (`proxy.ts`) with signed session cookies and protected `/api/admin` routes.
5. Established **production security headers** (CSP, HSTS, COOP) and **consent-gated Google Analytics 4** integration.
6. Delivered **technical SEO foundation**: dynamic sitemap, robots.txt, JSON-LD `JewelryStore` schema, per-page Open Graph, and generated OG images.
7. Configured **GitHub Actions CI** with parallel lint, typecheck, security audit, and production build verification.
8. Created **data-driven collection architecture** supporting 4 categories and 23 subsection pages from centralized TypeScript config.
9. Optimized performance via **dynamic imports**, `optimizePackageImports` , and **next/image** with responsive `sizes` and lazy loading.
10. Authored legal/compliance pages (privacy, refund, shipping, cookies) and cookie consent banner for launch readiness.

11.2 One-Line Summaries (Pick One for Resume Header)

A — Full-stack focus:
Full-stack developer — Next.js 16 luxury jewellery platform with Prisma lead CRM, secured admin APIs, and cinematic React motion.

B — Frontend focus:
Frontend engineer — React 19 / Next.js showcase site with GSAP + Framer Motion, Tailwind v4 design system, and 71-page static SEO architecture.

C — Performance & SEO focus:
Web engineer — High-performance marketing site (SSG, image optimization, CI) with JSON-LD local SEO for Rajasthan jewellery retail.

D — Security & infra focus:
Developer — Production-hardened Next.js app: CSP/HSTS headers, Upstash rate limits, Vercel deployment, and GitHub Actions quality gates.

E — Product/UX focus:
Product-minded developer — Lead-generation jewellery experience (WhatsApp, appointments, studio maps) without e-commerce checkout complexity.

11.3 Skills Tags for ATS

`Next.js` `React` `TypeScript` `Tailwind CSS` `Prisma` `PostgreSQL` `REST API` `GitHub Actions` `Vercel` `SEO` `JSON-LD` `Framer Motion` `GSAP` `Responsive Design` `Web Security` `Git`

---

12. Interview Preparation

12.1 HR / Behavioral

Q: Tell me about this project.
A: MB Jewellers is a premium showcase website for a family jewellery studio in Sikar, Rajasthan. It is not e-commerce—we drive private viewings and inquiries. I built it on Next.js 16 with static generation for about 71 routes, a custom motion stack, lead APIs with Prisma, and production security for Vercel deployment.

Q: What was your biggest challenge?
A: Balancing cinematic animations (Lenis + GSAP + Framer) with performance and accessibility. I used dynamic imports, `optimizePackageImports`, `prefers-reduced-motion` checks, and `gsap.context().revert()` cleanup to avoid scroll jank and memory leaks.

Q: How did you work with stakeholders?
A: Content is data-driven in TypeScript modules (`siteData`, `collectionPages`) so non-developers can update copy and imagery paths without touching components. Legal and policy pages use a shared `PremiumLegalPage` template.

12.2 React

Q: Why React 19 for this project?
A: React 19 is the paired version for Next.js 16.2.4. We use client components for interactivity and server components for metadata-heavy pages, reducing client JS on static marketing routes.

Q: How do you avoid hydration errors?
A: The `useClientMounted` hook gates browser-only UI. Forms avoid random IDs. Extension-safe patterns are used on inputs. Motion components that depend on `window` load with `dynamic(..., { ssr: false })`.

Q: Explain your component composition.
A: `PremiumSite` is the layout shell. Pages import focused components from `premium/pages`.

Shared motion hooks live in `premium/motion/premiumMotion.tsx`. Data is lifted to `app/lib/*` rather than prop drilling across deep trees.

12.3 Next.js

Q: App Router vs Pages Router?
A: This project uses App Router exclusively— `app/page.tsx`, colocated `layout.tsx`, Route Handlers in `app/api`, and special files like `sitemap.ts`, `robots.ts`, `opengraph-image.tsx`.

Q: How does static generation work here?
A: Most marketing pages are static. Dynamic segments implement `generateStaticParams` — products from `featuredPieces`, collection sections from `buildCollectionSectionParams()`, blog from `blogPosts`.

Q: What is `proxy.ts`?
A: Next.js 16 middleware convention (renamed from middleware). It protects `/admin` and `/api/admin` by verifying a signed session cookie before allowing access.

Q: How do you handle metadata?
A: Root metadata in `layout.tsx`; per-route `export const metadata` using `createPageMetadata()` for titles, descriptions, canonicals, and OG/Twitter cards.

12.4 JavaScript

Q: How do you validate API input without Zod in every route?
A: `validation.ts` provides typed parsers with manual checks—string normalization, max lengths, regex for email/phone, honeypot field. Zod is in dependencies for future consolidation.

Q: Explain the honeypot pattern.
A: Hidden `website` field—bots fill it; server returns `{ ok: true }` without storing lead if non-empty. Real users leave it blank.

Q: How does `lead-store` abstract storage?
A: Single API (`insertInquiry`, etc.) branches on `LEAD_STORAGE_MODE` —JSON file under `.data/` for local dev, Prisma for production Postgres.

12.5 TypeScript

Q: How is type safety enforced?
A: Strict TypeScript, shared types like `ProductItem`, `CollectionPageConfig`,

`ContactInquiryInput`. `as const` on config objects preserves literal types for slugs and sections.

Q: Example of a typed data contract?
A: `CollectionSubsection` requires `id`, `title`, `description`, `coverAlt`; optional `coverImage`. `buildCollectionSectionParams()` returns a typed array for `generateStaticParams`.

12.6 Tailwind CSS

Q: Why Tailwind v4?
A: Utility-first styling matches rapid luxury UI iteration. PostCSS plugin `@tailwindcss/postcss` integrates with Next.js. Custom tokens live in `globals.css` (premium theme variables).

Q: How do you keep UI consistent?
A: Reusable classes: `premium-glass-card`, `premium-eyebrow`, `site-px`. Components encapsulate repeated patterns (header, footer, legal page frame).

12.7 Animation

Q: Why both GSAP and Framer Motion?
A: Framer excels at React gesture-driven UI (magnetic buttons, layout animations). GSAP ScrollTrigger excels at scroll-scrubbed timelines and counters. Lenis normalizes scroll input for both.

Q: How do you prevent ScrollTrigger bugs on navigation?
A: `PremiumProviders` kills all triggers on unmount. Each hook uses `gsap.context()` scoped to a ref and calls `ctx.revert()` in the effect cleanup.

12.8 SEO

Q: How do you handle canonical URLs?
A: `siteConfig.url` from `NEXT_PUBLIC_SITE_URL` plus path in `createPageMetadata` → `alternates.canonical`.

Q: What schema.org types do you use?
A: `WebSite` and `JewelryStore` with address, hours, telephone, `priceRange`, and `sameAs` social profiles. FAQ schema helper exists for future `/faq` enrichment.

12.9 Deployment

Q: Describe your CI pipeline.
A: Three jobs on push/PR to master: quality (lint, typecheck, format), security (npm audit critical gate), build (Next.js build + verify script). Concurrency cancels stale runs.

Q: Vercel-specific considerations?
A: Enforce `NEXT_PUBLIC_SITE_URL` in production on Vercel (`VERCEL=1` check in `siteConfig`). Prisma `generate` runs on postinstall. Image optimization enabled when `NODE_ENV=production`.

12.10 Advanced

Q: How would you add true e-commerce later?
A: Prisma already models `Product` , `Category` , `Collection` . Would add cart session, payment provider (Razorpay), inventory flags, and separate checkout API routes—keeping marketing SSG and isolating transactional routes as dynamic server actions.

Q: How would you scale rate limiting globally?
A: Already supports Upstash Redis REST increment + TTL. Fallback in-memory `Map` is dev-only; production must set Upstash env vars.

Q: CSP blocks inline scripts—how do maps/GA work?
A: CSP explicitly allows `maps.googleapis.com` , `googletagmanager.com` , and `google-analytics.com` in script-src/connect-src/img-src. Tradeoff: `'unsafe-inline'` still required for some third-party embed patterns.

---

13. Portfolio Case Study

13.1 Problem

MB Jewellers needed a digital presence matching the quality of their physical studio—without implying online checkout. The site had to feel **luxury**, load fast on mobile networks in Rajasthan, rank for **local jewellery searches**, and reliably capture leads.

13.2 Solution

A **Next.js 16** showcase platform with:

- Cinematic homepage (`PremiumHome`) and data-driven collection hierarchy (23 subsections)
- Nine featured product stories with dedicated SEO URLs

- Lead APIs with validation, rate limits, and optional Postgres
- Admin dashboard behind signed-cookie auth
- Full legal/compliance and consent-gated analytics

13.3 Technical Highlights

- **71 static routes** at build for edge delivery
- **Triple motion stack** (Lenis + GSAP + Framer) with reduced-motion respect
- **JewelryStore JSON-LD** tied to real Sikar coordinates
- **CI/CD** on GitHub Actions with production verification script

13.4 Outcomes (Expected / Post-Launch)

- Improved discoverability for studio visits and WhatsApp inquiries
- Operational lead queue via admin dashboard and email notifications
- Foundation for future CMS/product database without redesign

13.5 Your Role (Customize for Portfolio)

Suggested framing: *Sole developer* or *Lead frontend developer* — designed architecture, implemented premium UI, built APIs and Prisma schema, configured CI and deployment documentation.

13.6 Visuals to Include in Portfolio Deck

1. Homepage hero screenshot (desktop + mobile)
2. Collections hub + gold subsection grid
3. Architecture diagram (section 4.7)
4. Lighthouse / CI green screenshot
5. Admin dashboard (blur PII)

---

14. Production Readiness Checklist

Area	Item	Status	Notes
Build	`npm run build` passes	✅	~71 routes
Build	`npm run verify:build`	✅	Post-build script
CI	Lint + typecheck + format	✅	GitHub Actions

CI	Prisma generate in CI	✅	
CI	npm audit critical gate	✅	High reported non-blocking
SEO	sitemap.xml	✅	Dynamic
SEO	robots.txt	✅	Blocks admin/api
SEO	Per-page metadata	✅	`createPageMetadata`
SEO	JSON-LD JewelryStore	✅	
SEO	opengraph-image	✅	Generated
SEO	Search Console submission	⬜	Post-deploy
Legal	Privacy, terms, refund, shipping, cancellation, cookie, disclaimer	✅	
Legal	Cookie consent banner	✅	
A11y	Skip to content	✅	
A11y	Landmarks + focus-visible	✅	
A11y	Full keyboard/contrast audit	⚠️	Manual
Security	CSP, HSTS, headers	✅	Production only
Security	Admin proxy auth	✅	`proxy.ts`
Security	Rate limit + honeypot	✅	
Security	Strong admin secrets in prod	⬜	Ops
Data	Postgres + migrations	⬜	If using postgres mode
Data	Upstash rate limit	⬜	Recommended prod
Email	Resend configured	⬜	
Analytics	GA4 ID + consent	⚠️	Component ready
Hosting	Vercel project + env vars	⬜	
DNS	Domain → Vercel SSL	⬜	
Content	Replace placeholder images	⚠️	Ongoing
Content	Proofread policies	⬜	
Testing	Forms E2E on production	⬜	
Testing	Unit/E2E automation	❌	Optional later
Perf	Lighthouse on prod URL	⬜	

Ops	Monitor Vercel logs	⬜	Post-launch

Overall launch readiness: ~**85%** (per internal audit)—remaining work is primarily operations, content assets, and production secrets.

---

15. Future Improvements

15.1 Product & UX

1. **Headless CMS** (Sanity/Contentful) for collections, blog, and testimonials—reduce TypeScript-only content edits
2. **Real product photography pipeline** replacing Unsplash placeholders
3. **Breadcrumbs** and related products on `/products/[slug]`
4. **Sticky mobile conversion bar** for appointments
5. **Google Reviews embed** on testimonials page

15.2 Technical

1. **Prisma migrations** committed; seed script for staging
2. **Zod schemas** shared between client forms and API routes
3. **bcrypt** password hashing for admin (replace env plain compare)
4. **E2E tests** (Playwright) for contact flow and admin login
5. **Remove legacy** `DPHomePage` / unused Header components
6. **FAQPage schema** wired on `/faq`
7. **BreadcrumbList** JSON-LD on collection/product routes
8. **WebP/AVIF** asset pipeline for `/public/pics`
9. **Service worker** or PWA enhancements beyond basic manifest
10. **i18n** proper locale routes instead of client-only Google Translate

15.3 Performance

1. Hero video: shorter loop, WebM alternate, poster-first LCP
2. Reels section behind `prefers-reduced-motion` and intersection observer
3. Vercel Speed Insights + Core Web Vitals alerts
4. Fix Next.js LCP warning on header logo (`priority` tuning)

15.4 Business / Analytics

1. Meta Pixel / conversion API (if marketing expands)
2. CRM webhook (HubSpot/Zoho) from lead APIs
3. Appointment Google Calendar sync (schema field `googleCalendarEventId` exists)
4. Inventory-backed product pages from Prisma `Product` model

15.5 Security & Compliance

1. CSP `report-uri` / Reporting API endpoint
2. Rotate secrets playbook documented
3. Periodic dependency updates (Next.js patch cadence)
4. DPA documentation if storing EU visitor data

Appendix A — Route Inventory (Approximate)

Category	Count
Core marketing (home, about, contact, etc.)	~18
Legal policies	7
Collection hubs	4
Collection subsections	23
Product detail (SSG)	9
Blog index + articles	4
Admin + login	2
System (sitemap, robots, manifest, OG, 404)	~5
API routes	~8 (dynamic, not in static 71)

Total static/SSG pages at build: ~71

Appendix B — Featured Products (9)

#	Name	Material	Slug
1	Emerald Diamond Set	Diamond	`emerald-diamond-set`

2	Aurora Halo Drops	Diamond	`aurora-halo-drops`
3	Luna Cluster Pendant	Diamond	`luna-cluster-pendant`
4	Heritage Filigree Chokar	Gold	`heritage-filigree-chokar`
5	Royal Gold Chokar Set	Gold	`royal-gold-chokar-set`
6	Regal Coin Chokar	Gold	`regal-coin-chokar`
7	Moonlight Silver Hoops	Silver	`moonlight-silver-hoops`
8	Arctic Line Pendant	Silver	`arctic-line-pendant`
9	Silver Dew Bracelet	Silver	`silver-dew-bracelet`

Appendix C — API Endpoints

Method	Path	Purpose
POST	`/api/contact`	Contact inquiry
POST	`/api/appointments`	Appointment booking
POST	`/api/newsletter`	Newsletter subscribe
POST	`/api/admin/login`	Admin session
POST	`/api/admin/logout`	Clear session
PATCH	`/api/admin/inquiries/[id]`	Update lead status
PATCH	`/api/admin/appointments/[id]`	Update appointment status

Appendix D — Related Documentation

- `README.md` — Setup and scripts
- `PRODUCTION_READINESS.md` — Audit checklist (May 2026)
- `VERCEL_DEPLOY.md` — Deployment steps
- `LAUNCH_CHECKLIST.md` — Launch tasks
- `.env.example` — Environment template

This document describes the MB Jewellers codebase as maintained in the `chetansaini0/mb_jewellers` repository. Re-run `npm run build` and update route counts after major routing changes.